

Face Recognition System - User Manual

Version: 1.0.1 | Date: 2026-03-12

1. Purpose

This manual explains operation of the distributed Face Recognition System, including architecture, roles, real-time behavior, security controls, and troubleshooting.

2. System Architecture

The solution uses a client-server architecture with centralized backend services and distributed surveillance clients.

2.1 Components

- Surveillance Post PC(s): React frontend used by operators.
- Administration Post PC: Django API, WebSocket gateway, PostgreSQL, and model server.
- Model Server: Face recognition processing service.

2.2 Data and Control Flow

1. Frontend user authenticates through Django.
2. Django stores JWT access/refresh tokens in HTTP-only cookies.
3. REST requests use cookie credentials and CSRF protection.
4. WebSocket authentication uses JWT cookies.
5. Guest presence socket updates online state.
6. Admin notification socket receives guest connect/disconnect events.
7. Video stream consumer sends frame and detection payloads in real time.

2.3 Logical Diagram

```
SURVEILLANCE POST(S)
  React Client (UI, AuthContext, LangContext)
    | HTTP + WebSocket
    v
ADMINISTRATION POST
  Django + DRF + Channels (port 8000)
    |-- PostgreSQL (records, users, roles, presence)
    |-- Model Server (face recognition service)
```

2.4 Current Code Anchors

- `ADMINISTRATION_POST/DJANGO_SERVER/server/auth_views.py`: auth, role and guest management endpoints.
- `ADMINISTRATION_POST/DJANGO_SERVER/server/urls.py`: REST route map.
- `ADMINISTRATION_POST/DJANGO_SERVER/server/routing.py`: WebSocket route map.
- `ADMINISTRATION_POST/DJANGO_SERVER/server/consumers.py`: stream and notification consumers.
- `SURVEILLANCE_POST/REACT_CLIENT/src/services/api.js`: API/CSRF client behavior.
- `SURVEILLANCE_POST/REACT_CLIENT/src/context/AuthContext.jsx`: frontend auth state and guest presence socket lifecycle.

3. Roles and Permissions

3.1 Admin

- Full access to management pages and dashboards.
- Can create grant tokens, assign roles, and manage guest accounts.
- Receives real-time guest presence notifications.

3.2 Guest

- Limited application routes only.
- Can use operational pages allowed by policy.
- Sends presence heartbeat over WebSocket while connected.

4. Authentication and Security

- JWT access/refresh in HTTP-only cookies.
- Refresh rotation + blacklist support.
- CSRF bootstrap endpoint and request header injection for unsafe methods.
- Login lockout and DRF throttle scopes for abuse resistance.
- Camera passwords encrypted at rest.

5. API and WebSocket Endpoints

5.1 Auth APIs

`GET /api/auth/csrf/, POST /api/auth/login/, POST /api/auth/logout/, POST /api/auth/refresh/, GET /api/auth/me/, POST /api/auth/grant-token/, POST /api/auth/register-with-token/, POST /api/auth/grant-role/, GET/POST /api/auth/guests/, PATCH/DELETE /api/auth/guests/{user_id}/`

5.2 Main APIs

`/api/compagnies/, /api/persons/, /api/cameras/, /api/spectacles/, /api/rentrees/, /api/status/`

5.3 WebSocket Endpoints

`/ws/video/stream/, /ws/face/recognize/, /ws/presence/guest/, /ws/admin/notifications/`

6. Daily Operations

1. Start administration services from `ADMINISTRATION_POST/START_ADMIN_SERVICES.ps1`.
2. Start surveillance client from `SURVEILLANCE_POST/START_SURVEILLANCE_CLIENT.ps1`.
3. Admin logs in, opens Access Control, provisions users, and monitors guest presence.
4. Guest logs in and uses allowed operational routes.
5. Use logout before ending a shift.

7. Important Configuration Notes

Backend production settings are controlled in `ADMINISTRATION_POST/DJANGO_SERVER/.env` (hosts, CORS, CSRF trusted origins, JWT cookie settings, logout/throttles).

Current frontend API base is `http://localhost:8000/api` in `src/services/api.js`. For distributed deployment, set API/WS base URLs to the administration post IP.

8. Troubleshooting

- Login logout: wait for lock duration or clear cache according to admin procedure.
- Guest online state missing: verify `/ws/presence/guest/` connectivity.
- No admin notifications: verify `/ws/admin/notifications/` connectivity and role.
- Frontend API errors: check firewall, admin host reachability, CORS and CSRF trusted origins.

9. Localization

The UI supports English and Arabic using `LangContext`. The language toggle affects user-facing labels and status text.